REPORT

Forest Fire Detection & Spread Prediction System

Team Members:

Don Varkey Joshy
John K Simson
Avadhanam Hasini

Milestone – 1

Problem Statement

Forest fires are increasing due to rising temperatures and changing climate conditions. Traditional fire detection methods rely on satellite monitoring or manual inspection, which can result in delayed detection. This delay allows small fires to spread rapidly, causing severe environmental damage, loss of biodiversity, and economic loss. Therefore, a real-time monitoring system is required to detect fire early and predict its spread effectively.

Overview of System Design

The first milestone focuses on designing an IoT-based forest fire detection and prediction system. The system uses sensors to continuously monitor environmental conditions such as temperature, humidity, gas levels, and flame presence. An ESP32 microcontroller collects and processes sensor data, which is then transmitted to the cloud using WiFi. The cloud platform stores and visualizes this data, enabling real-time monitoring and analysis. This design provides a scalable and efficient solution for early fire detection and prediction.

System Architecture Description

The system follows a layered architecture consisting of four main layers:

1. Sensing Layer

This layer includes sensors such as:

- DHT11 (Temperature & Humidity)
- MQ-2 (Gas/Smoke detection)
- Flame Sensor (Fire detection)

These sensors collect environmental data continuously.

2. Processing Layer

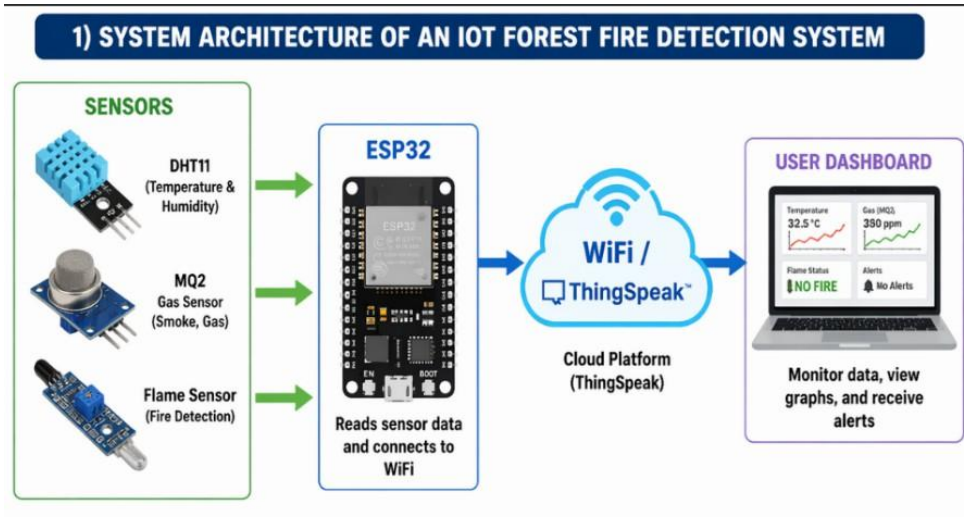
The ESP32 microcontroller reads sensor data, processes it, and applies fire detection logic. It also calculates a risk score for fire spread prediction based on environmental conditions.

3. Cloud/Storage Layer

The processed data is sent to the ThingSpeak cloud platform. The data is stored with timestamps, enabling continuous logging and future analysis.

4. Visualization Layer

The data is displayed using graphs and dashboards on ThingSpeak. This allows users to monitor fire conditions and trends remotely in real time.

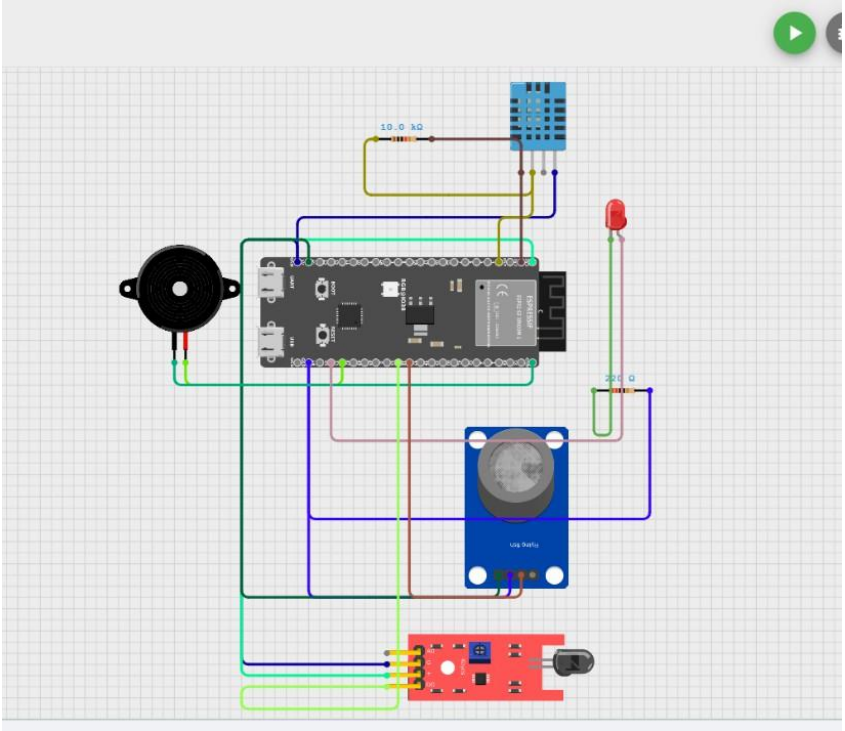


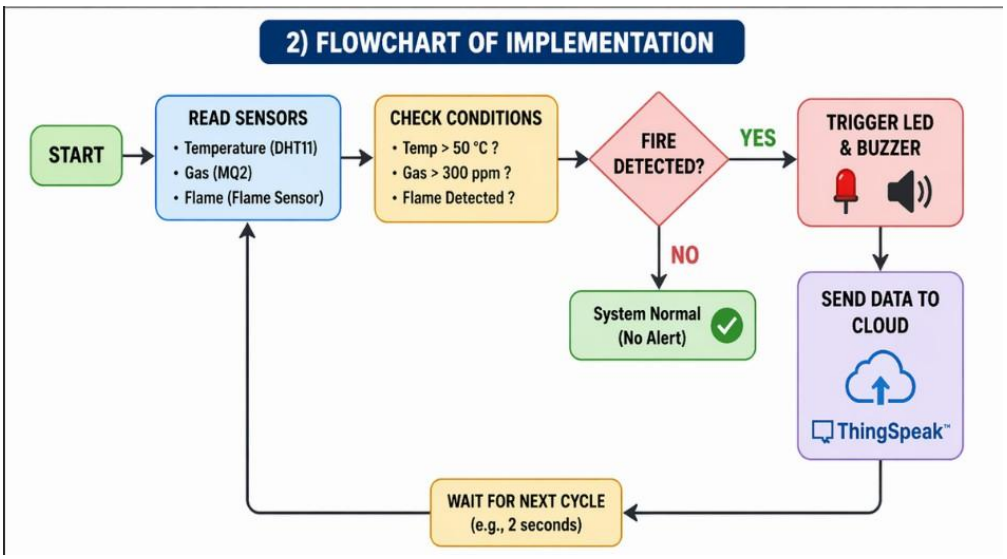
Simulation Setup and Implementation

The system was simulated using Cirkit Designer to validate connections and functionality before hardware implementation. All components including ESP32, DHT11, MQ-2, and flame sensor were connected virtually. Proper pin configurations, power supply connections, and communication methods were ensured.

The simulation allowed real-time monitoring of sensor values and helped verify correct wiring, sensor behavior, and data acquisition.

Fire detection logic was implemented, where alerts are triggered when abnormal conditions are detected. This step ensured the system design was functional and reliable.





Cloud Integration using ThingSpeak

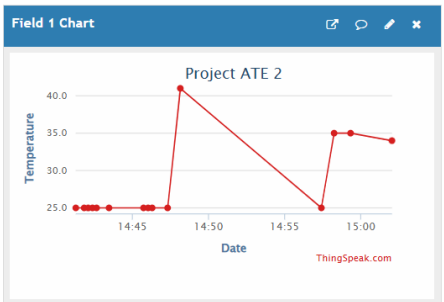
To enable real-time monitoring, the system integrates with the ThingSpeak cloud platform. The ESP32 sends sensor data such as temperature, humidity, gas levels, and flame status to ThingSpeak using WiFi. Each parameter is assigned to specific fields in the channel.

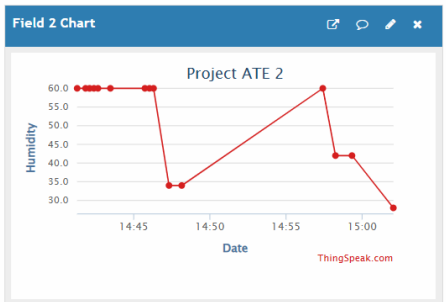
ThingSpeak stores the data and provides graphical visualization, allowing users to observe trends and variations. The platform also supports data export for further analysis and machine learning.

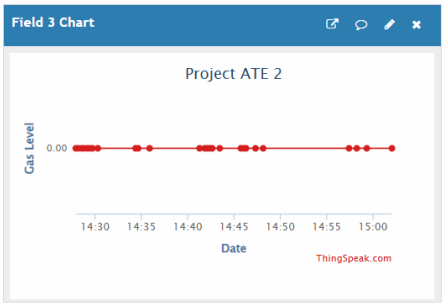
This integration enables:

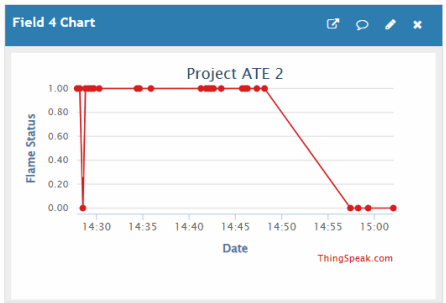
- Remote monitoring

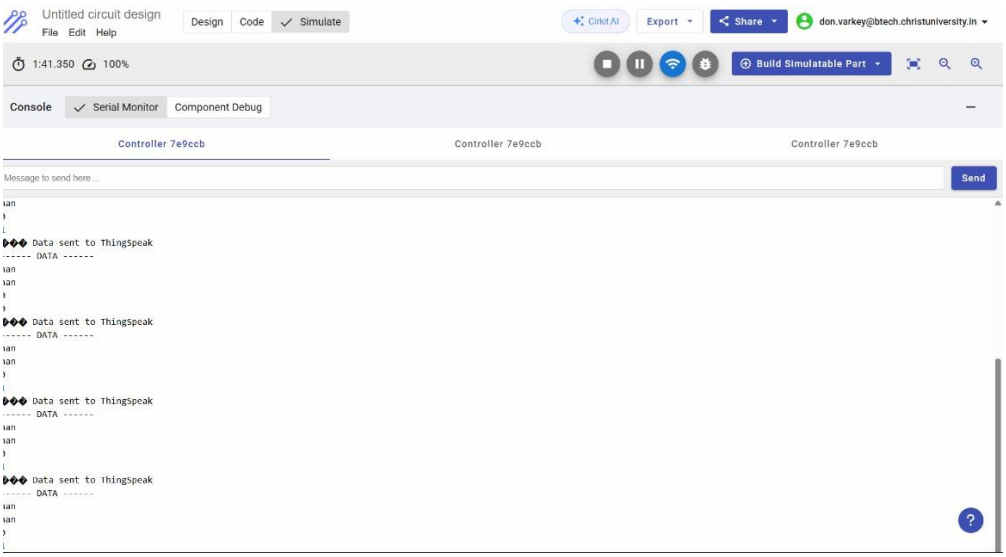
- Real-time data visualization
- Continuous data storage











Outcome

At the end of this milestone, a complete system architecture has been successfully designed. The simulation setup has been implemented and validated, and real-time data transmission to ThingSpeak has been achieved. The system is capable of detecting fire conditions and provides a foundation for fire spread prediction. This milestone establishes the base for further development, including machine learning models and advanced analytics in future stages.

Milestone – 3

Live Data Simulation & Model Integration

Overview

Milestone 3 integrates live sensor data with a trained machine learning model to generate real-time early fire-risk predictions. The ESP32 microcontroller continuously reads environmental sensor values (temperature, humidity, gas/smoke levels, and flame detection) and uploads them to the ThingSpeak cloud platform every 15 seconds. A Python-based prediction pipeline fetches this live data, enriches it with real-time weather API readings from OpenWeatherMap, fuses the two data sources, and feeds the combined feature vector into a trained Gradient Boosting Classifier to output a fire spread risk level.

Objectives

- Simulate real-time environmental sensor data and upload it to the ThingSpeak cloud.
- Fetch live weather data (temperature, humidity, wind speed) from the OpenWeatherMap API.
- Integrate live data with the trained ML model to generate early fire-risk predictions.
- Fuse IoT sensor readings and API weather values into a unified feature vector.
- Classify each reading into one of four fire-risk levels: LOW, MEDIUM, HIGH, or FIRE.
- Save prediction results to CSV and Excel files for analysis and reporting.

System Data Flow

The end-to-end pipeline follows a sequential flow from sensor hardware to ML prediction output:

```
ESP32 + Sensors  →  ThingSpeak Cloud  →  Python Script  →  OpenWeatherMap
                                                  API
 →  Data Fusion (Sensor + API)  →  Gradient Boosting Model  →  Fire Risk
                                                 Level
```

Part A: IoT Sensor Simulation & Cloud Upload

Hardware Components Used

Component	Description / Role
ESP32 (Wokwi / Cirkit)	Microcontroller — reads sensors, connects to WiFi, uploads to ThingSpeak

DHT11	Temperature & humidity sensor (Field 1 & 2 on ThingSpeak)
MQ-2 (Digital)	Gas/smoke sensor — binary output: 0 (no gas) / 1 (gas detected)
Flame Sensor (Digital)	Flame detection — binary output: 0 (no flame) / 1 (flame detected)
LED (Pin 20)	Visual alert indicator — HIGH when gas or flame is detected
ThingSpeak Channel	Channel ID: 3333848 — stores timestamped sensor readings

ESP32 Arduino Code Walkthrough

The Arduino sketch running on the ESP32 performs the following tasks in sequence:

Library Imports & Pin Configuration

```
#include <WiFi.h>
#include <ThingSpeak.h>
#include <DHT.h>

#define DHTPIN    4      // DHT11 data pin
#define DHTTYPE   DHT11
#define MQ2_PIN   36     // MQ-2 digital output pin
#define FLAME_PIN 21     // Flame sensor digital output pin
#define LED_PIN   20     // Alert LED pin
```

WiFi & ThingSpeak Initialization

```
const char* ssid     = "CirkitWifi";
const char* password = "";
unsigned long channelID  = 3333848;
const char* writeAPIKey  = "0R8ZCZW6P0FU7CLO";

void setup() {
  dht.begin();
  pinMode(MQ2_PIN,   INPUT);
  pinMode(FLAME_PIN, INPUT);
  pinMode(LED_PIN,   OUTPUT);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); }
  ThingSpeak.begin(client);
}
```

Main Loop — Sensor Read, Alert, & Upload

```
void loop() {
  float temp = dht.readTemperature();
  float hum  = dht.readHumidity();
  if (isnan(temp) || isnan(hum)) {
    temp = random(25, 40);   // fallback if DHT11 read fails
    hum  = random(40, 80);
  }
  int gas   = digitalRead(MQ2_PIN);
  int flame = digitalRead(FLAME_PIN);

  // LED alert logic
```

```
  if (flame == 1 || gas == 1) digitalWrite(LED_PIN, HIGH);
  else                        digitalWrite(LED_PIN, LOW);

  // Upload to ThingSpeak
  ThingSpeak.setField(1, temp);
  ThingSpeak.setField(2, hum);
  ThingSpeak.setField(3, gas);
  ThingSpeak.setField(4, flame);
  ThingSpeak.writeFields(channelID, writeAPIKey);
  delay(15000);  // ThingSpeak rate limit: 1 upload per 15s
}
```

ThingSpeak Field Mapping

ThingSpeak Field	Sensor	Data / Unit
Field 1	DHT11	Temperature (°C)
Field 2	DHT11	Humidity (%)
Field 3	MQ-2 Gas Sensor	Gas/Smoke (Binary: 0 or 1)
Field 4	Flame Sensor	Flame Detected (Binary: 0 or 1)

Live ThingSpeak Channel Output

The following screenshot shows the live ThingSpeak channel graphs capturing real-time sensor data from the ESP32. All four fields — Temperature (Field 1), Humidity (Field 2), Gas/Smoke (Field 3), and Flame (Field 4) — are actively updated every 15 seconds.

Figure: ThingSpeak live channel — Forest Fire sensor readings (Fields 1–4)

Part B: ML Model Training (train_model.py)

Before live prediction can occur, a Gradient Boosting Classifier is trained on a synthetic dataset calibrated to real Uttarakhand 2024 wildfire conditions. This trained model is then serialized to disk for use by the live prediction pipeline.

Synthetic Dataset Generation

The training dataset is generated using four fire-scenario classes with the following distribution:

Scenario	Weight	Temp Range	Gas / Flame
Normal (LOW)	35%	14–26°C	0 / 0
Medium Risk	30%	28–36°C	Random / 0
High Risk	20%	36–46°C	1 / 0
FIRE	15%	46–65°C	1 / 1

Feature Engineering

Each row in the dataset contains raw sensor values and derived engineered features:

- Final_Temp — average of sensor temperature and API temperature
- Final_Hum — average of sensor humidity and API humidity
- Dryness_Index — Final_Temp minus Final_Hum (key fire-risk indicator; >28 is dangerous for Uttarakhand)
- Wind_Speed — from OpenWeatherMap API (km/h)
- Gas — binary MQ-2 output (0 = no gas, 1 = gas detected)
- Flame — binary flame sensor output (0 = no flame, 1 = flame detected)
- Temp_Diff — absolute difference between sensor and API temperature readings
- Hum_Diff — absolute difference between sensor and API humidity readings

Model Configuration

```
model = GradientBoostingClassifier(
    n_estimators  = 150,   # number of boosting stages
    learning_rate = 0.1,   # shrinks contribution of each tree
    max_depth     = 4,     # maximum depth per tree
    random_state  = 42
)
model.fit(X_train, y_train)
```

Label Classification Rules

Label	Condition
FIRE	Flame == 1 AND Gas == 1 (both sensors active simultaneously)
HIGH	Dryness_Index > 28 AND Wind_Speed > 28 km/h

MEDIUM	Dryness_Index > 12 OR Wind_Speed > 15 km/h
LOW	None of the above conditions are met

Part C: Live Prediction Pipeline (predict_live.py)

The prediction script runs in a loop, fetching new ThingSpeak data every 15 seconds (synchronized with the ESP32 upload interval), enriching it with live weather data, and running the Gradient Boosting model.

Data Sources

Source	API / Platform	Fields Fetched
IoT Sensor (ESP32)	ThingSpeak REST API	Temperature, Humidity, Gas, Flame
Live Weather	OpenWeatherMap API	Temp, Humidity, Wind Speed (km/h)

Data Fusion Logic

Sensor and API values are fused by averaging to produce the final feature inputs for the model:

```
final_temp    = (sensor_temp + api_temp) / 2
final_hum     = (sensor_hum  + api_hum)  / 2
dryness_index = final_temp - final_hum
temp_diff     = abs(sensor_temp - api_temp)   # disagreement metric
hum_diff      = abs(sensor_hum  - api_hum)    # disagreement metric
```

Sample Prediction Output

Each prediction cycle produces a structured console output block showing raw values, comparison metrics, fused values, and the final fire risk decision:

```
╔═══════════════════════════════════════════════════════════════╗
╠═══ PREDICTION CYCLE #1 ═══                                   ═╣
╠═══════════════════════════════════════════════════════════════╣
  Sensor Data : ThingSpeak LIVE — channel 3333848
  Weather     : OpenWeatherMap LIVE (Nainital)
╠═══════════════════════════════════════════════════════════════╣
  ═══ RAW VALUES ═══
  Sensor Temperature  : 38.2°C
  Sensor Humidity     : 18.5%
  API Temperature     : 36.0°C
  API Humidity        : 22.0%
  Wind Speed          : 32.4 km/h
  Gas (MQ2)           : HIGH  [Binary: 1]
  Flame Detected      : NO    [Binary: 0]
╠═══════════════════════════════════════════════════════════════╣
  ═══ FUSED VALUES (fed into ML model) ═══
  Final Temperature   : 37.1°C
  Final Humidity      : 20.25%
  Dryness Index       : 16.85
╠═══════════════════════════════════════════════════════════════╣
```

```
──── DECISION OUTPUT ────
Fire Alert          :  NORMAL
Fire Spread Level   :  HIGH
```

Prediction Output Files

After completing all prediction cycles, results are saved in two formats for analysis and reporting:

- forest_fire_predictions.csv — comma-separated values, suitable for Excel, pandas, and Power BI.
- forest_fire_predictions.xlsx — formatted Excel workbook with all cycle results including timestamp, sensor readings, gas/flame values, fire alert, and spread level.

Outcome

At the end of Milestone 3, the following deliverables were achieved:

- Real-time sensor data is successfully uploaded to ThingSpeak every 15 seconds from the simulated ESP32 environment.
- Live ThingSpeak data is fetched and combined with OpenWeatherMap weather data in Python.
- A Gradient Boosting Classifier generates fire spread risk predictions (LOW / MEDIUM / HIGH / FIRE) for each cycle.
- A hard safety rule ensures that simultaneous Gas=1 and Flame=1 always triggers a FIRE DETECTED alert, regardless of model output.
- All prediction results are exported to CSV and Excel files for downstream analysis in Power BI (Milestone 4).

This milestone establishes the live inference foundation of the system — bridging IoT hardware simulation with machine learning to deliver continuous, automated fire-risk assessment.